

Duplicados retrasados

El problema y cómo evitarlo

Capa de transporte

Capítulo 3

¿Por qué importan los duplicados?

¿Pueden viajar segmentos duplicados de un emisor a un receptor?

- **Sí.** Por ejemplo, cuando se pierde un *ack* y un segmento se retransmite, o cuando expira el temporizador por congestión.

Exigencia

- No se pueden entregar segmentos duplicados a la capa de aplicación.

Consecuencia

- El receptor debe poder decidir, para cada segmento que llega, si es duplicado o no.

¿Cómo detectar duplicados eficientemente?

✗ Comparar bit a bit

Solución inviable

- Requiere almacenar todos los segmentos previos.
- Costo de memoria y cómputo prohibitivo.

✓ Números de secuencia

Solución viable

- Numerar cada segmento con un **n° de secuencia**.
- Dos segmentos con n° distinto son distintos.
- Funcionaría perfecto si los números fueran de tamaño arbitrario.

Pero el espacio de secuencia es finito

¿Por qué?

- Los segmentos tienen **longitud máxima** → el n° de secuencia debe entrar en un campo del encabezado.
- Ese campo tiene **longitud fija** → solo hay un número finito de valores posibles.

Consecuencia

- Los números de secuencia se **reutilizan**. Numerar los segmentos solo con eso ya no alcanza.

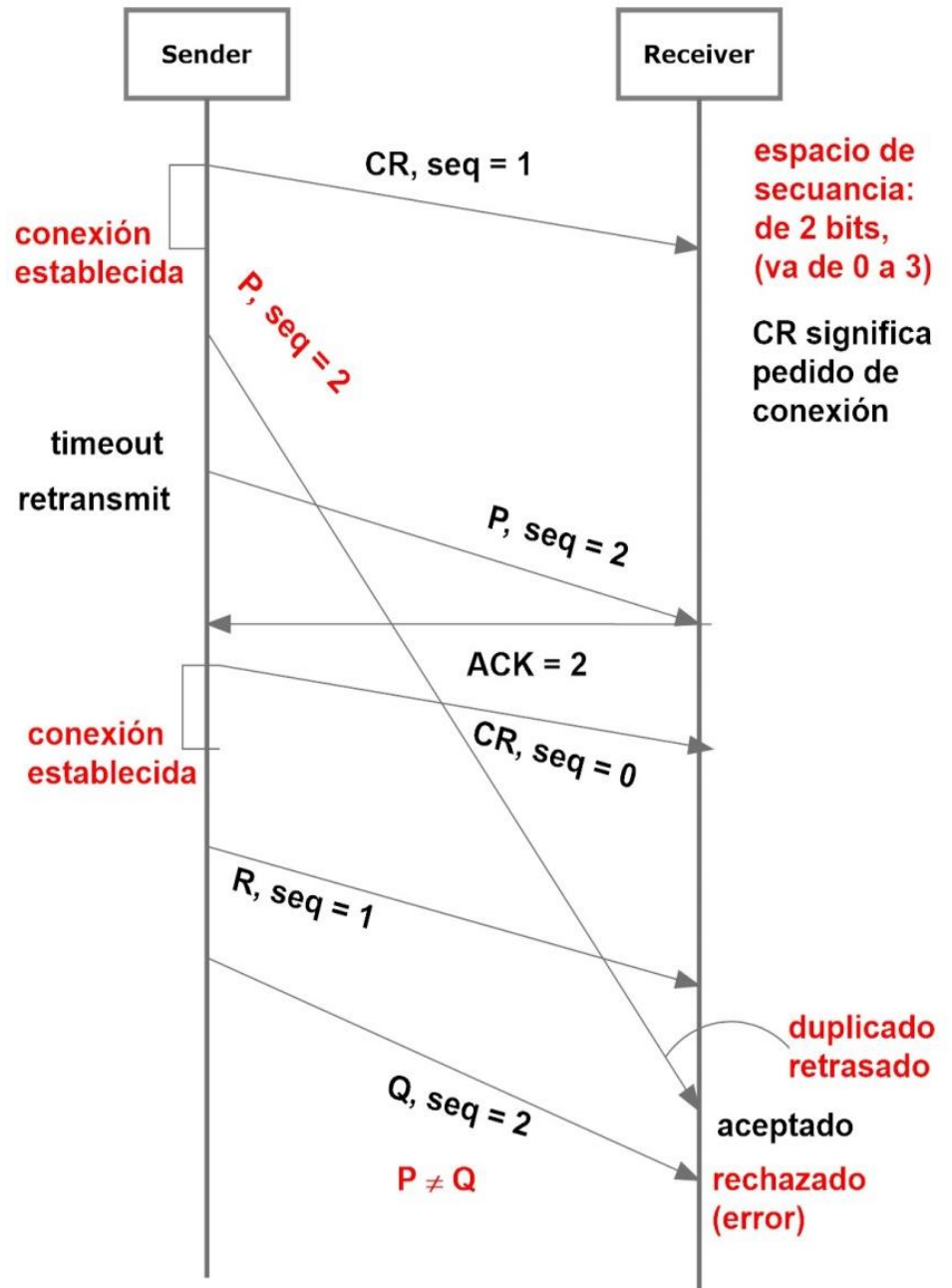
A — Dentro de una conexión | B — Entre conexiones

DOS ESCENARIOS DONDE APARECEN PROBLEMAS

Escenario B Entre conexiones

En el ejemplo:

- Espacio de secuencia de 2 bits (0–3).
- Antes de la 2da conexión se libera la 1ra.
- La 2da conexión llega rápido al n° 2.
- Un duplicado retrasado de la 1ra conexión con n° 2 aún vive en la red.
- **Es aceptado por error** en la 2da conexión.



Escenario A

Dentro de una conexión

El espacio de secuencia se cicla

- Los n° van 0→1→2→3, vuelven a 0 y suben hasta 3 de nuevo.
- Un segmento con n° **3** queda demorado y llega cuando ya pasó el segundo **3**.

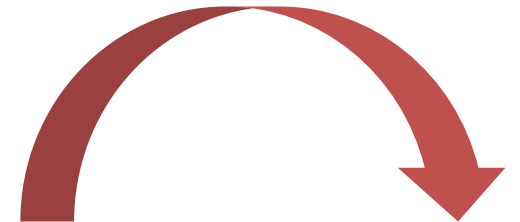
Resultado

- El receptor lo acepta como nuevo: el espacio de secuencia se reutilizó y el duplicado vivió demasiado en la red.

n° de secuencia (2 bits → 0,1,2,3 que se reciclan)



espacio se reutiliza



duplicado retrasado del primer "3"

La idea unificadora: limitar el tiempo de vida

Idea

- Asegurar que ningún paquete viva más de *T segundos* en la red.

Aplica a:

- Paquetes de datos.
- Retransmisiones.
- Confirmaciones de recepción (acks).

Eliminar paquetes viejos que sigan circulando hace que ambos escenarios sean manejables.

Solución para el Escenario A (dentro de una conexión)

Regla

- El origen no debe **reutilizar un n° de secuencia dentro de T segundos** (tiempo de vida del paquete).

¿Por qué funciona?

- Cuando se llega a un n° ya usado, ese segmento viejo ya murió hace tiempo en la red.
- El espacio de secuencia debe ser **lo bastante grande** para que no se cicle dentro de T segundos.

Ejercicio: dimensionar el espacio de secuencia

Enunciado

Una conexión usa segmentos de **1500 B**, transmite a **10 Mbps**, y el tiempo de vida máximo de segmento es **$T = 30$ s**. ¿De cuántos bits debe ser el n° de secuencia?

Planteo

- Necesitamos que en **T** segundos **no se repita** ningún n° de secuencia.
- Hay que contar **cuántos segmentos como máximo** se pueden transmitir en **T** segundos.

Ejercicio: cálculo y respuesta

Cuenta

- Bits transmitidos en **T**: $10^7 \text{ bits/s} \times 30 \text{ s} = 3 \times 10^8 \text{ bits}$
- Bits por segmento: $1500 \times 8 = 12.000 \text{ bits}$
- Segmentos máximos en **T**: $3 \times 10^8 / 1,2 \times 10^4 = 25.000 \text{ segmentos}$
- Bits necesarios: $\log_2(25.000) \approx 14,6$

Respuesta: 15 bits.

Solución para el Escenario B (entre conexiones)

El problema

- Si la nueva conexión empieza en **0**, puede chocar con duplicados de la conexión anterior.

Idea

- Cada conexión elige un **n° inicial de secuencia distinto** para no entrar en el rango de los duplicados de conexiones anteriores.

Implementación 1: n° inicial aleatorio

Libro Comer — algunas implementaciones de TCP

Cómo

- Al crear una nueva conexión, cada extremo elige un **n° aleatorio de 32 bits**. Ese n° pasa a ser el n° inicial de secuencia para los datos.

¿Por qué tiende a funcionar?

- La probabilidad de que un duplicado retrasado caiga en el rango de la nueva conexión es **baja** gracias a la elección aleatoria.

Implementación 2: n° inicial atado al reloj

Libro Tanenbaum

Cómo

- Cada host tiene un **contador binario** que se incrementa en intervalos uniformes.
- El n° inicial de cada nueva conexión se toma del **valor actual del reloj**.
- Los relojes de los hosts no necesitan estar sincronizados.

¿Por qué funciona?

- El reloj avanza monotónicamente: el n° inicial es siempre **posterior** al usado por la conexión previa, fuera del rango de sus duplicados.

Ejemplo: TCP con esquema basado en reloj

Implementación de TCP descrita por Tanenbaum

- Pulso de reloj cada **4 μ s**.
- Tiempo de vida máximo de paquete: **T = 120 s**.
- N° de secuencia: **32 bits**.

*El reloj corre lo bastante rápido como para que cada nueva conexión abra en un valor distinto, y lo bastante lento como para no agotar los 32 bits dentro de **T** segundos.*

En síntesis

Dentro de una conexión

Causa

- El espacio de secuencia se cicla.

Solución

- Hacer el espacio de secuencia **lo bastante grande** como para que un n° no se reutilice antes de T segundos.

Entre conexiones

Causa

- Una nueva conexión puede caer en n° ya en uso por una conexión previa.

Solución

- Elegir un **n° inicial distinto** en cada conexión.
- Aleatorio (Comer) o atado al reloj (Tanenbaum).